

Hybrid CTA Approach Based on Pretrained Embeddings Similarity and Tree Ensemble Classification

Panagiotis Koletsis¹, Christos Panagiotopoulos¹, Georgios Th. Papadopoulos¹,
Iraklis Varlamis¹ and Vasilis Efthymiou¹

¹Harokopio University of Athens (HUA), Omirou 9, Tavros, Athens, 17778, Greece

Abstract

The rapid growth of data generation has increased the need for automated methods that can interpret and organize raw tabular data, enabling effective integration and analysis. This paper addresses the Column Type Annotation (CTA) task of the SemTab challenge, which aims to identify the semantic type of tabular data columns. We propose a hybrid approach that combines embedding-based cosine similarity, domain and cell-value constraints, and supervised classification. Specifically, we construct prototype embeddings for each column type and perform similarity-based matching, while unresolved cases are further classified using a CatBoost model restricted to the top- k candidate classes ranked by cosine similarity. Since each stage of the pipeline is explicitly defined, the proposed approach is highly interpretable and transparent. Evaluation on three SemTab 2023 datasets demonstrates the competitiveness of our approach in terms of Micro-F1 score, while achieving the highest precision among the evaluated methods. Finally, we present additional experiments exploring alternative configurations that did not yield further performance improvements but provide useful insights into the impact of different design choices and the effectiveness of the proposed components.

1. Introduction

Since the widespread adoption of the internet, organizations have been collecting large amounts of tabular data [1]. However, many of these data remains poorly documented or semantically ambiguous, which limits its immediate use. Before tabular data can be effectively integrated, searched, or analysed, its columns must be assigned meaningful semantic descriptions. Manually performing this task is costly and difficult to scale, creating a clear need for accurate automated annotation methods.

To address those issues, the SemTab challenge [2] provides benchmark datasets for evaluating systems that interpret tabular data using knowledge graphs (KG) such as Wikidata and DBpedia. These datasets contain noisy and incomplete tables with limited metadata, thereby reflecting many of the difficulties encountered in real-world data-integration settings. The main tasks provided by SemTab are described next. **Column Type Annotation (CTA)**: Identify the KG class that best represents the values of a target column. This is the focus of the present work. **Column Property Annotation (CPA)**: Identify the KG property that describes the relationship between two columns. **Cell Entity Annotation (CEA)**: Match an entity mentioned in a table cell to its corresponding entity in the KG. **Topic Detection (TD)**: Identify the KG class that best describes the overall topic of a table.

The CTA task has received considerable attention in recent years, attracting strong participation from numerous research teams. This has led to the development of highly effective approaches, with some participants achieving great results on the task. These approaches are mostly utilizing API-based searches on popular KGs [3], training table interpretation transformer models [4] or utilizing pretrained LLMs [5]. Although these approaches can achieve strong performance, they may require expensive external queries, substantial resources, or models whose decision process is difficult to inspect. Moreover, most systems are not publicly available, which limits reproducibility and direct comparison.

In this work, we investigate whether a lightweight and fully local hybrid pipeline can achieve competitive CTA performance while retaining a transparent and traceable inference process. Our approach

OM 2026: The 21st International Workshop on Ontology Matching, co-located with ISWC 2026, October 25, 2026, Bari, Italy

✉ pkoletsis@hua.gr (P. Koletsis); chris.panagiotopoulos@hua.gr (C. Panagiotopoulos); g.th.papadopoulos@hua.gr

(G. Th. Papadopoulos); varlamis@hua.gr (I. Varlamis); vefthym@hua.gr (V. Efthymiou)

🌐 https://github.com/panagiotis-koletsis/cta_full_pipeline (P. Koletsis)

🆔 0009-0005-2904-7923 (P. Koletsis); 0009-0000-8195-3898 (C. Panagiotopoulos); 0000-0003-1686-421X

(G. Th. Papadopoulos); 0000-0002-0876-8167 (I. Varlamis); 0000-0002-0683-030X (V. Efthymiou)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

combines embedding-based cosine similarity, column-value evidence, domain-based candidate restrictions, and supervised classification, achieving competitive performance or even surpassing state-of-the-art (SOTA) approaches. At the same time, it is fully open source, with a permissive license, unlike most other methods which are not publicly available.

Specifically, we construct prototype embeddings for each column type and initially rank candidate classes according to their cosine similarity with the target column. High-confidence cases are resolved directly, while ambiguous columns are processed using cell-value evidence and a CatBoost classifier constrained to the top-k similarity candidates. This staged design combines semantic and statistical evidence while reducing the effective classification space.

The main contributions of this work are the following:

- A lightweight prototype-based CTA method that represents columns using pretrained embeddings and matches them to class prototypes through cosine similarity.
- An embedding-based classification methodology, using statistical and semantic constraints.
- An open-source unified pipeline that combines all the aforementioned components, available on github: https://github.com/panagiotis-koletsis/cta_full_pipeline
- An empirical analysis of alternative model configurations that provides useful insight into the contribution and limitations of the individual pipeline components.

2. Related Work

In this section, we explore several important works related to the CTA task, which can be categorized into four main groups: lookup-based, model-based, LLM-based, and hybrid approaches.

Lookup Based. Most recent CTA systems are lookup-based, relying on knowledge graph (KG) search (e.g., Wikidata). MTab [6] dominated the SemTab 2021 challenge by introducing a system that leverages a local Wikidata dump for entity retrieval and annotation. To improve heuristic matching, the authors incorporated several preprocessing steps and employed multiple search strategies, including keyword search, fuzzy search based on the Levenshtein distance, and an aggregation search that combines the results of the previous two methods to enhance matching accuracy. The next year, after years of improvements DAGOBAB [7] dominated. DAGOBAB, similarly to MTab, relies on KG search for entity matching. In addition to incorporating a fuzzy matching mechanism, it introduces an alias-based search strategy to improve entity retrieval. It also introduces an improved entity scoring algorithm for more accurate matching and header prediction. Similarly, Kepler-aSI [8], after preprocessing and data cleaning, utilizes SPARQL queries to perform the matching.

Model Based. TorchicTab [4] finetunes the pretrained language model DODUO [9] to perform the CTA task. This model works by breaking large tables into smaller sub-tables through random row and column sampling. After tokenization, these sub-tables are parsed through a transformer network capable of capturing context from both rows and columns, and at the end, the final prediction is determined using a majority voting strategy. In contrast, DREIFLUSS [10] follows a simpler methodology, relying on a logistic regression model for classification.

LLM Based. Recent works in SemTab have explored the effectiveness of Large Language Models (LLMs) for table interpretation tasks. TSOTSA [11] investigated zero-shot and few-shot approaches by leveraging a GPT-3 model. After preprocessing, the authors converted the input tables into JSON format and provided them to a carefully designed prompt developed through prompt engineering techniques.

Hybrid. SemTex [3], on the other hand, follows a hybrid approach that combines KG search with gradient boosting-based prediction. Entity disambiguation is performed through the Bing API search and the generation of candidate entities. When the KG search returns multiple possible candidates, the final entity prediction is determined using a CatBoost classifier. MUT2KG [12] introduces a neuro-symbolic framework that integrates symbolic KG querying with deep neural vector embeddings, leveraging anchor nodes to align extracted subgraph concepts with cell and column embeddings.

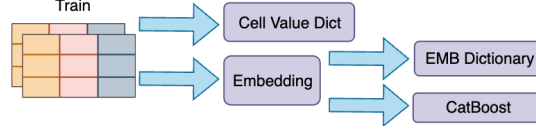


Figure 1: Training Phase: Generation process of auxiliary files from the training and validation sets.

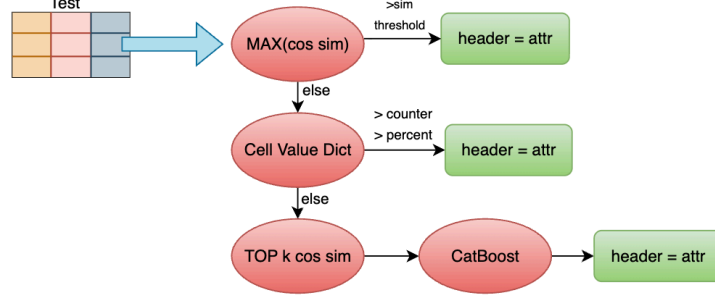


Figure 2: Inference Phase: Exploiting the files generated in training to classify the columns of a given table.

3. Methodology

This section describes in detail all the components of the proposed two-phase pipeline, consisting of training and inference phases, as shown in Figures 1 and 2, respectively. Let $C = (v_1, \dots, v_n)$ denote a target column, $y \in \mathcal{Y}$ its column-type label, and $f_\theta(\cdot) \in \mathbb{R}^d$ the selected pretrained embedding model. The same preprocessing and embedding procedure is applied during training and inference.

3.1. Training Phase

First, we describe the components of the training phase, as shown in Figure 1. No test labels or test-table values are used to construct the auxiliary dictionaries, tune the model, or the decision thresholds.

Embedding Prototypes Dictionary. All the embeddings of the pipeline are extracted by isolating each target column and its cell values. After excluding empty, None, and other irrelevant values, we follow one of the two approaches below for generating column embeddings: (i) **CONCAT**: We concatenate all cell values into a single string using “|” as a separator and obtain the corresponding embedding by passing the resulting string to the embedding model. For the sequence of retained values $\tilde{C} = (\tilde{v}_1, \dots, \tilde{v}_m)$, this representation is defined as $\mathbf{e}_{\text{CONCAT}}(C) = f_\theta(\tilde{v}_1 \parallel \dots \parallel \tilde{v}_m)$, where $\parallel$ denotes concatenation with “|” as the separator. (ii) **AVG**: We compute the embedding of each individual cell value and represent the column by the average of all cell embeddings. Formally, $\mathbf{e}_{\text{AVG}}(C) = \frac{1}{m} \sum_{j=1}^m f_\theta(\tilde{v}_j)$.

For efficiency, this deterministic rule uses the first 40 rows of each column instead of a random sample, or the entire column. Additionally, inputs exceeding 7,500 tokens are truncated to fit the embedding model’s context window. We iterate over the validation set and construct a dictionary that stores a representative embedding for each header class. The representative embedding is computed as the average of the embeddings of all columns belonging to the corresponding header class. Let $\mathcal{D}_y^{\text{val}}$ denote the validation columns annotated with type y . The prototype of type y is $\mathbf{p}_y = \frac{1}{|\mathcal{D}_y^{\text{val}}|} \sum_{C_i \in \mathcal{D}_y^{\text{val}}} \mathbf{e}(C_i)$. A separate prototype dictionary is constructed for each dataset and embedding configuration.

Cell Value Dictionary. In this step, we construct an index (a python dictionary) from the cell values that appear in the training and validation sets. For each cell value, the dictionary records the column headers in which it appears along with the number of occurrences under each header, excluding empty, null, or other irrelevant values. For a retained value v and class y , let $n(v, y)$ denote the number of occurrences of v in development columns of class y . We store its total support, dominant class, and class purity as $n(v) = \sum_{y \in \mathcal{Y}} n(v, y)$, $y^*(v) = \arg \max_{y \in \mathcal{Y}} n(v, y)$, and $q(v) = \frac{n(v, y^*(v))}{n(v)}$. Dictionary keys are generated using exactly the same cell preprocessing as the embedding stage.

Top- k & CatBoost. We augment the training and validation datasets with an additional column containing the embedding of the corresponding table column referenced in each row. This auxiliary feature is used as an input to the CatBoost [13] model during training. More precisely, CatBoost is trained on pairs $(\mathbf{e}(C_i), y_i)$, treating the d embedding dimensions as numerical features. Let $P_\phi(y | \mathbf{e}(C))$ denote the class probability produced by the fitted model. In essence, the CatBoost model is trained to perform column classification by predicting the corresponding header class based on the extracted column embeddings. The final prediction of the model is defined as the highest-scoring model output that is also included in the Top- k candidates from the “cosine similarity” ranking-process.

Domain Restrictions. Based on the training and validation sets, we construct a domain dictionary. This dictionary contains, for each table domain, all attributes that appear as column headers within that domain. This is made possible by the fact that the table name typically indicates its corresponding domain. Let $d(T)$ denote the domain associated with table T , and let $\mathcal{A}_{d(T)} \subseteq \mathcal{Y}$ be the set of column types observed for that domain in the training and validation sets. At every inference stage, candidate classes are restricted to $\mathcal{A}_{d(T)}$. This is beneficial because the reduced search space introduces additional constraints on the final prediction. This component is utilized across all stages of the pipeline.

3.2. Inference Phase

After constructing the Embedding Prototypes Dictionary, the Cell Value Dictionary, and training the CatBoost model, we perform inference as illustrated in Figure 2.

First, we extract the embedding of the target column following the same Embedding Extraction process as in the training phase and compute its cosine similarity with all prototype embeddings stored in the Embedding Prototypes Dictionary. For each $y \in \mathcal{A}_{d(T)}$, the similarity score is $s_y(C) = \cos(\mathbf{e}(C), \mathbf{p}_y) = \frac{\mathbf{e}(C)^\top \mathbf{p}_y}{\|\mathbf{e}(C)\|_2 \|\mathbf{p}_y\|_2}$. Let $y_s = \arg \max_{y \in \mathcal{A}_{d(T)}} s_y(C)$. If the highest similarity score exceeds the empirically selected threshold $\tau_s = 0.92$, the corresponding class is directly assigned as the prediction.

If no prototype satisfies this threshold, we consult the Cell Value Dictionary. A prediction is made only when a cell value (i) appears more than five times in the column and (ii) is associated with the same class in more than 92% of its occurrences, indicating a strong correspondence between the value and the column type. Specifically, if $m_C(v)$ is the frequency of value v within the target column, a dictionary match is eligible when $m_C(v) > 5$, $q(v) > 0.92$, and $y^*(v) \in \mathcal{A}_{d(T)}$. The exact rule for aggregating or breaking ties among multiple eligible values is stated in the released code and identified by the repository commit used for the reported experiments.

If neither of the above steps produces a confident prediction, the target column embedding is passed to the trained CatBoost model. Let $\mathcal{K}_7(C)$ denote the seven classes with the highest cosine-similarity scores among the domain-compatible candidates. The final prediction is selected as the highest-probability class among the CatBoost outputs that also appear in the Top-7 classes ranked by cosine similarity in the first step, and is evaluated using exact matching. The fallback decision is therefore $\hat{y}(C) = \arg \max_{y \in \mathcal{K}_7(C)} P_\phi(y | \mathbf{e}(C))$. The complete inference procedure is thus a three-stage cascade consisting of high-confidence prototype matching, high-purity cell-value matching, and Top- k -constrained CatBoost reranking.

4. Experiments

Hardware-Software. All the experiments were performed on a single GPU (RTX 3080 Ti with 12GB VRAM). The access to the models was achieved through Ollama (v.0.18.1)¹.

Models. For the embedding models, Nomic-Embed-Text-v2-MoE (approximately 300M active parameters)² and Snowflake-Arctic-Embed2 (approximately 568M parameters)³ were evaluated. For classification, the CatBoost model [13] was used.

¹ollama: <https://ollama.com>

²Nomic: <https://ollama.com/library/nomic-embed-text-v2-moe>

³Snowflake: <https://ollama.com/library/snowflake-arctic-embed2>

Table 1

Statistics of Semtab 2023 datasets [14].

Dataset	Data Points	Domains	Classes
Schema.org R1	~56k	7	40
Schema.org R2	~118k	14	80
DBpedia R2	~87k	14	46

Table 2

Evaluation and comparison with SOTA. Our methods are below the dashed lines.

Dataset	Method	Macro_F1	Micro_F1	Precision	Recall
Schema.org R1	Kepler-aSI [8]	-	0.3640	0.3430	-
Schema.org R1	Nomic-notAVG	0.8535	0.8644	0.8915	0.8547
Schema.org R1	Snowflake-AVG	0.8568	0.8719	0.8936	0.8616
Schema.org R2	TSOTSA [11]	-	0.3705	0.5601	-
Schema.org R2	Anu	-	0.5872	0.7048	-
Schema.org R2	Kepler-aSI [8]	-	0.3372	0.3601	-
Schema.org R2	TorchicTab [4]	-	0.8966	0.8980	-
Schema.org R2	MUT2KG [12]	-	0.3201	0.7922	-
Schema.org R2	DREIFLUSS [10]	-	0.3804	0.5767	-
Schema.org R2	Nomic-notAVG	0.8559	0.8570	0.8830	0.8580
Schema.org R2	Snowflake-AVG	0.8857	0.8831	0.9129	0.8848
DBpedia R2	TSOTSA [11]	-	0.3906	0.6150	-
DBpedia R2	Anu	-	0.6150	0.5330	-
DBpedia R2	Kepler-aSI [8]	-	0.4611	0.4983	-
DBpedia R2	TorchicTab [4]	-	0.8972	0.9093	-
DBpedia R2	MUT2KG [12]	-	0.3330	0.8196	-
DBpedia R2	DREIFLUSS [10]	-	0.4097	0.6114	-
DBpedia R2	Nomic-notAVG	0.8721	0.8590	0.8979	0.8746
DBpedia R2	Snowflake-AVG	0.8861	0.8781	0.9186	0.8883

Table 3

Execution time of each module in seconds. CB: CatBoost

Method	Dataset	Prot Dict	Cell Dict	Emb calc	CB train	Inference	Total
Nomic-notAVG	Schema.org R1	112	44	3947	150	51	4304
Snowflake-AVG	Schema.org R1	107	118	9261	202	112	9800
Nomic-notAVG	Schema.org R2	86	272	8257	342	114	9071
Snowflake-AVG	Schema.org R2	229	272	20023	490	280	21294
Nomic-notAVG	DBpedia R2	53	195	6096	200	63	6607
Snowflake-AVG	DBpedia R2	131	195	14654	249	145	15374

Datasets. We evaluate our approach on the SemTab 2023 datasets for the CTA task, specifically the Round 1 and Round 2 SOTAB Schema.org-based datasets and the Round 2 DBpedia-based dataset [14]. Information about each dataset can be found in the Table 1.

Evaluation metrics. We employ the standard metrics used in the evaluation of SemTab, i.e., Macro_Precision, Macro_Recall, Macro_F1, and Micro_F1, as well as execution time in seconds. To ensure that all methods are evaluated consistently, the SemTab challenge provides an evaluator Python script, which computes these metrics using the scikit-learn library.

Hyperparameter selection. The CatBoost model used to evaluate our pipeline was tuned with Optuna [15] using the Tree-structured Parzen Estimator (TPE) sampler over 50 optimization trials. TPE is a Bayesian optimization method that exploits the results of previous trials to guide the search toward promising hyperparameter configurations. The optimized model improved the micro-F1 score by approximately 2%. The hyperparameter search space was: depth: integer [4,8], learning_rate: log-uniform [0.01,0.3], l2_leaf_reg: log-uniform [1.0,10.0], bagging_temperature: uniform [0.0,1.0], random_strength: uniform [0.0,10.0], early_stopping: 30, iterations: 500.

Results Discussion. The Snowflake embedding-based approach consistently outperformed the Nomic embedding-based approach across all evaluated metrics as shown in Table 2, albeit with a no-

table increase in execution time as shown in Table 3. This increase can be attributed to two main factors. First, the Snowflake embedding model contains nearly twice as many parameters as the Nomic embedding model, making inference inherently more computationally expensive. Second, the superior-performing Snowflake approach employs the AVG strategy, in which a representative embedding for each column is computed by generating an embedding for every cell and then averaging them. In contrast, the Nomic approach uses the Concat strategy, which concatenates all cell values into a single string and computes a single embedding for the resulting text. On the **R1 Schema.org** dataset, Snowflake achieved only a 0.87% improvement over Nomic, while increasing execution time by more than 120%. This was the only dataset for which the additional computational cost does not appear to be justified by the relatively small performance gain. Compared to the reported SOTA results on Kepler-aSI, both of our approaches achieved substantially better performance across the reported metrics. On the **R2 Schema.org** dataset, Snowflake-AVG outperformed Nomic-notAVG by 3.0% in terms of Micro-F1. Compared with the reported state-of-the-art results, TorchicTab achieved a 1.5% higher Micro-F1 score than our approach, placing our method second overall. However, our method outperformed TorchicTab in precision by 1.6%, achieving the highest precision among all evaluated methods. Similarly, on the **R2 DBpedia** dataset, Snowflake-AVG outperformed Nomic-notAVG by 2.2% in terms of Micro-F1. Although TorchicTab achieved the highest Micro_F1 score, placing our method second overall, Snowflake-AVG achieved the highest precision among all evaluated methods, highlighting the strong ability of the proposed approach to produce accurate positive predictions. Finally, the proposed approach offers improved interpretability, as its predictions are based on explicit intermediate components rather than relying solely on a black-box transformer model.

Additional experiments that did not yield improved results. (i) In addition to CatBoost, we evaluated XGBoost [16] and LightGBM [17] as alternative classifiers; however, both models achieved slightly lower performance. (ii) At the end of the embedding, we appended one more boolean feature, indicating whether the column corresponds to a functional property or not, i.e., whether the column has duplicate values after removing nulls. (ii) At the end of the embedding, we appended an embedding of the entire table. The goal was to provide the model with global table context awareness, which typically helps improve final header prediction. (iv) At the end of the embedding, we appended an embedding calculated by another embedding model (i.e., Snowflake and Nomic embeddings). The goal was to capture contextual information of the column that the primary model may have missed.

5. Conclusion

In this work, we presented a hybrid approach for the CTA task, combining embedding-based similarity, structural constraints, and supervised classification. Evaluation on three SemTab 2023 datasets showed competitive Micro-F1 performance and the highest precision among the evaluated methods, demonstrating the effectiveness of combining semantic and statistical information for table interpretation. Future work will focus on improving the model by exploring new approaches and combining effective components. Potential directions include fine-tuning the embeddings for the CTA task and investigating more advanced similarity-based retrieval methods beyond standard cosine similarity.

Acknowledgments

This work has received funding from the EU Horizon Europe research and innovation programme through the GANNDALF project (GA No. 101167951).

Declaration on Generative AI

During the preparation of this work, the authors used ChatGPT and Claude in order to: Grammar, spelling check, paraphrasing and expediting code development. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the publication’s content.

References

- [1] H. Ekbja, M. Mattioli, I. Kouper, G. Arave, A. Ghazinejad, T. Bowman, V. R. Suri, A. Tsou, S. Weingart, C. R. Sugimoto, Big data, bigger dilemmas: A critical review, *Journal of the Association for Information Science and Technology* 66 (2015) 1523–1545. doi:<https://doi.org/10.1002/asi.23294>.
- [2] E. Jiménez-Ruiz, O. Hassanzadeh, V. Efthymiou, J. Chen, K. Srinivas, Semtab 2019: Resources to benchmark tabular data to knowledge graph matching systems, in: *The Semantic Web - 17th International Conference, ESWC 2020, Heraklion, Crete, Greece, May 31-June 4, 2020, Proceedings*, volume 12123 of *Lecture Notes in Computer Science*, Springer, 2020, pp. 514–530. doi:10.1007/978-3-030-49461-2_30.
- [3] E. G. Henriksen, A. M. Khorsid, E. Nielsen, A. M. Stück, A. S. Sørensen, O. Pelgrin, Semtex: A hybrid approach for semantic table interpretation., in: *SemTab@ ISWC, 2023*, pp. 38–49.
- [4] I. Dasoulas, D. Yang, X. Duan, A. Dimou, Torchictab: Semantic table annotation with wikidata and language models, in: *Proceedings of the Semantic Web Challenge on Tabular Data to Knowledge Graph Matching, SemTab 2023, co-located with the 22nd International Semantic Web Conference, ISWC 2023, Athens, Greece, November 6-10, 2023*, volume 3557 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023, pp. 21–37.
- [5] J. P. Bikim, C. A. A. Ymele, A. Jiomekong, A. Oelen, G. Rabby, J. D’Souza, S. Auer, Leveraging GPT models for semantic table annotation, in: *Proceedings of the Semantic Web Challenge on Tabular Data to Knowledge Graph Matching co-located with the 23rd International Semantic Web Conference (ISWC 2024), Baltimore, USA, November 13, 2024*, volume 3889 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2024, pp. 43–53.
- [6] P. Nguyen, I. Yamada, N. Kertkeidkachorn, R. Ichise, H. Takeda, Semtab 2021: Tabular data annotation with mtab tool, in: *Proceedings of the Semantic Web Challenge on Tabular Data to Knowledge Graph Matching co-located with the 20th International Semantic Web Conference (ISWC 2021), Virtual conference, October 27, 2021*, volume 3103 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2021, pp. 92–101.
- [7] V. Huynh, Y. Chabot, T. Labbé, J. Liu, R. Troncy, From heuristics to language models: A journey through the universe of semantic table interpretation with DAGOBAB, in: *Proceedings of the Semantic Web Challenge on Tabular Data to Knowledge Graph Matching, SemTab 2022, co-located with the 21st International Semantic Web Conference, ISWC 2022, Virtual conference, October 23-27, 2022*, volume 3320 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2022, pp. 45–58.
- [8] W. Baazouzi, M. Kachroudi, S. Faiz, Kepler-asi at semtab 2021., in: *SemTab@ ISWC, 2021*, pp. 54–67.
- [9] Y. Suhara, J. Li, Y. Li, D. Zhang, c. Demiralp, C. Chen, W.-C. Tan, Annotating columns with pre-trained language models, in: *Proceedings of the 2022 International Conference on Management of Data, SIGMOD ’22, Association for Computing Machinery, 2022*, p. 1493–1503. doi:10.1145/3514221.3517906.
- [10] V. R. Parmar, A. Algergawy, Dreifluss: A minimalist approach for table matching., in: *SemTab@ ISWC, 2023*, pp. 50–60.
- [11] J. P. Bikim, C. A. A. Ymele, A. Jiomekong, A. Oelen, G. Rabby, J. D’Souza, S. Auer, Leveraging gpt models for semantic table annotation., in: *SemTab@ ISWC, 2024*, pp. 43–53.
- [12] S. Mehryar, R. Celebi, Semantic annotation of tabular data for machine-to-machine interoperability via neuro-symbolic anchoring., in: *SemTab@ ISWC, 2023*, pp. 61–71.
- [13] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, A. Gulin, Catboost: unbiased boosting with categorical features, *Advances in neural information processing systems* 31 (2018).
- [14] K. Korini, R. Peeters, C. Bizer, Sotab v2 for semtab 2023, 2023. doi:10.5281/zenodo.8422037.
- [15] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, ACM, 2019, pp. 2623–2631. doi:10.1145/3292500.3330701.

- [16] T. Chen, C. Guestrin, Xgboost: A scalable tree boosting system, in: Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining, 2016, pp. 785–794.
- [17] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, Lightgbm: A highly efficient gradient boosting decision tree, Advances in neural information processing systems 30 (2017).